

Zagrożenia dla aplikacji webowych OWASP Top 10

Czym jest OWASP?

- OWASP – Open Worldwide Application Security Project
- OWASP Foundation to organizacja non-profit działająca na rzecz poprawy bezpieczeństwa oprogramowania założona 1 grudnia 2001 r.
- Tworzy społecznościowe projekty open source (kod, dokumentacja, standardy)
- Posiada ponad 250 lokalnych oddziałów na całym świecie
- Skupia dziesiątki tysięcy członków
- Organizuje uznane na świecie szkolenia i konferencje

Czym jest OWASP Top 10?

To zestawienie 10 najważniejszych zagrożeń bezpieczeństwa aplikacji webowych. Lista pokazuje najczęstsze i najgroźniejsze podatności, pomaga programistom i firmom je rozumieć i eliminować.

OWASP Top 10 jest aktualizowany co kilka lat. Zmiany wynikają z:

- rozwoju technologii (np. mikroserwisy, API)
- nowych typów ataków
- lepszych danych statystycznych

2013

- A1 – Injection
- A2 – Broken Authentication and Session Management
- A3 – Cross-Site Scripting (XSS)
- A4 – Insecure Direct Object References
- A5 – Security Misconfiguration
- A6 – Sensitive Data Exposure
- A7 – Missing Function Level Access Control
- A8 – Cross-Site Request Forgery (CSRF)
- A9 – Using Known Vulnerable Components
- A10 – Unvalidated Redirects and Forwards

2017

- A1 - Injection
- A2 - Broken Authentication
- A3 - Sensitive Data Exposure
- A4 - XML External Entities (XXE)
- A5 - Broken Access Control
- A6 - Security Misconfiguration
- A7 - Cross-Site Scripting (XSS)
- A8 - Insecure Deserialization
- A9 - Using Components with Known Vulnerabilities
- A10 – Insufficient Logging & Monitoring

2021

- A1 - Broken Access Control
- A2 - Cryptographic Failures
- A3 - Injection
- A4 - Insecure Design
- A5 - Security Misconfiguration
- A6 - Vulnerable and Outdated Components
- A7 - Identification and authentication failures
- A8 - Software and Data Integrity Failures
- A9 - Security Logging and Monitoring Failures
- A10 - Server-Side Request Forgery



OWASP Top 10 2025

- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging and Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

Broken Access Control

To sytuacja, w której aplikacja nie pilnuje, kto ma prawo do jakiej operacji lub danych. Skutek bywa prosty: ktoś może podejrzeć cudze dane, zmienić je, usunąć albo wykonać akcję zarezerwowaną dla wyższych uprawnień.

Broken Access Control (40 CWEs)

- **CWE-284 (Improper Access Control)** — System nie ogranicza poprawnie dostępu do zasobów, przez co użytkownicy mogą wykonywać operacje, do których nie powinni mieć uprawnień.
- **CWE-285 (Improper Authorization)** — Aplikacja nie sprawdza właściwie, czy użytkownik ma prawo wykonać daną akcję.
- **CWE-862 (Missing Authorization)** — Brak mechanizmu autoryzacji dla danej operacji, co pozwala każdemu ją wykonać.
- **CWE-863 (Incorrect Authorization)** — Autoryzacja istnieje, ale działa błędnie i pozwala ominąć ograniczenia.
- **CWE-918 (Server-Side Request Forgery, SSRF)** — Aplikacja wykonuje żądania do innych systemów na podstawie danych użytkownika, co pozwala atakującemu uzyskać dostęp do wewnętrznych zasobów.

Broken Access Control

- Wymuszanie kontroli dostępu po stronie serwera
Kontrola dostępu powinna być realizowana wyłącznie w zaufanym kodzie serwera lub w serverless API, a nie w logice po stronie klienta. Dzięki temu atakujący nie może zmodyfikować warunku sprawdzającego uprawnienia ani metadanych używanych do decyzji.
- Domyślna odmowa dostępu
Dla wszystkich zasobów innych niż publiczne najlepiej stosować zasadę „deny by default”. Oznacza to, że użytkownik otrzymuje dostęp tylko wtedy, gdy został on jawnie przyznany, co zmniejsza ryzyko przypadkowego ujawnienia danych.
- Jednokrotna implementacja mechanizmów uprawnień
Kontrola dostępu powinna być zdefiniowana raz i wielokrotnie używana w całej aplikacji, zamiast powtarzania jej w wielu miejscach. Taki model ogranicza niespójności i zmniejsza szansę, że jakaś ścieżka zostanie pominięta albo zaimplementowana błędnie.
- Ograniczanie użycia CORS
Warto minimalizować Cross-Origin Resource Sharing, bo zbyt szeroka konfiguracja może niepotrzebnie rozszerzyć powierzchnię ataku. Najbezpieczniej dopuszczać tylko te źródła, które rzeczywiście są wymagane przez aplikację.
- Unieważnianie sesji i krótkie życie tokenów
Stateful session identifiers powinny być unieważniane po stronie serwera po wylogowaniu, a stateless JWT powinny mieć krótki czas życia. Przy dłużej ważności JWT warto stosować refresh tokeny oraz standardy OAuth do odwoływania dostępu, żeby skrócić okno, w którym przejęty token może zostać wykorzystany.

Security Misconfiguration

Chodzi o błędną konfigurację systemu, aplikacji albo usług chmurowych z punktu widzenia bezpieczeństwa. Zagrożenie polega na tym, że włączone są zbędne funkcje, pozostają domyślne hasła, ujawniają się zbyt szczegółowe błędy albo działają niebezpieczne ustawienia w komponentach.

Security Misconfiguration (16 CWEs)

- **CWE-489 (Active Debug Code)** — W kodzie produkcyjnym pozostają mechanizmy debugowania, które mogą ujawniać wrażliwe informacje lub umożliwiać manipulację.
- **CWE-526 (Exposure of Sensitive Information Through Environmental Variables)** — Wrażliwe dane (np. klucze API) są dostępne przez zmienne środowiskowe i mogą zostać ujawnione.
- **CWE-547 (Use of Hard-coded Security-relevant Constants)** — W kodzie na stałe zapisane są wartości istotne dla bezpieczeństwa (np. klucze), które można łatwo odczytać.
- **CWE-614 (Sensitive Cookie in HTTPS Session Without 'Secure' Attribute)** — Cookie może być przesyłane także przez niezabezpieczone połączenie, mimo że powinno być tylko przez HTTPS.
- **CWE-1004 (Sensitive Cookie Without 'HttpOnly' Flag)** — Cookie jest dostępne z poziomu JavaScript, co umożliwia jego kradzież np. przez XSS.

Security Misconfiguration

- Minimalna platforma bez zbędnych komponentów
Należy instalować tylko te funkcje, frameworki i elementy dokumentacji, które są rzeczywiście potrzebne. Im mniej zbędnych części w systemie, tym mniejsza powierzchnia ataku i mniej miejsc, które trzeba aktualizować i zabezpieczać.
- Regularna aktualizacja konfiguracji i poprawek
Częścią zarządzania poprawkami powinien być przegląd i aktualizacja ustawień związanych z notami bezpieczeństwa, łataniami i zmianami konfiguracyjnymi. Trzeba też kontrolować uprawnienia do zasobów chmurowych, takich jak bucket S3, bo błędna polityka dostępu często prowadzi do wycieków danych.
- Segmentacja architektury aplikacji
Aplikację warto projektować tak, aby jej komponenty były od siebie logicznie i sieciowo odseparowane, na przykład przez segmentację, konteneryzację albo grupy bezpieczeństwa w chmurze. Taka izolacja utrudnia ruch boczny atakującego i ogranicza skutki kompromitacji jednego elementu.
- Ręczna weryfikacja, jeśli brak automatyzacji
Jeżeli danej kontroli nie da się zautomatyzować, trzeba ją sprawdzać ręcznie przynajmniej raz w roku. To ważne, bo konfiguracje i zależności zmieniają się z czasem, a stare założenia bezpieczeństwa mogą przestać być aktualne.

Software Supply Chain Failures

To awarie lub kompromitacje w łańcuchu dostarczania oprogramowania: podczas budowania, dystrybucji lub aktualizacji.

Ryzyko pojawia się, gdy aplikacja korzysta z podatnych, nieutrzymywanych albo zmodyfikowanych bibliotek, narzędzi i zależności, które wchodzą do systemu z zewnątrz.

Software Supply Chain Failures (6 CWEs)

- **CWE-1104 (Use of Unmaintained Third Party Components)** — Aplikacja korzysta z bibliotek, które nie są już utrzymywane i mogą zawierać znane podatności.
- **CWE-1329 (Reliance on Component That is Not Updateable)** — System zależy od komponentu, którego nie da się zaktualizować, co utrudnia usunięcie podatności.
- **CWE-1357 (Reliance on Insufficiently Trustworthy Component)** — Wykorzystywany komponent pochodzi z niepewnego źródła lub nie jest godny zaufania.
- **CWE-1395 (Dependency on Vulnerable Third-Party Component)** — Aplikacja używa komponentu z już znaną podatnością.

Software Supply Chain Failures

- Ograniczanie powierzchni ataku przez usuwanie zbędnych elementów
Trzeba usuwać nieużywane biblioteki, funkcje, pliki, dokumentację i komponenty, które nie są potrzebne do działania systemu. Im mniej zależności i zbędnych zasobów, tym mniej miejsc, które mogą zostać wykorzystane przez atakującego.
- Ciągła inwentaryzacja wersji komponentów i zależności
Dobłą praktyką jest stałe zbieranie informacji o wersjach komponentów po stronie klienta i serwera, razem z ich zależnościami. Do tego można używać narzędzi typu OWASP Dependency Track, OWASP Dependency Check czy retire.js, aby szybciej wykrywać przestarzałe lub podatne elementy.
- Pobieranie komponentów wyłącznie z zaufanych źródeł
Komponenty powinny pochodzić tylko z oficjalnych, zaufanych źródeł i być pobierane przez bezpieczne połączenia. Podpisane paczki dodatkowo zmniejszają ryzyko włączenia zmodyfikowanego, złośliwego pakietu do aplikacji.
- Świadomy dobór wersji zależności
Wersje bibliotek trzeba wybierać celowo, zamiast aktualizować je bez kontroli lub z przypadku. Aktualizacja powinna następować wtedy, gdy istnieje konkretny powód, na przykład poprawka bezpieczeństwa, wymóg funkcjonalny albo usunięcie ryzyka związanego ze starą wersją.
- Regularna aktualizacja narzędzi developerskich
CI/CD, IDE i inne narzędzia deweloperskie powinny być aktualizowane na bieżąco. Przestarzałe narzędzia mogą same stać się wektorem ataku albo wprowadzać nieaktualne, niebezpieczne konfiguracje do procesu wytwarzania oprogramowania.

Cryptographic Failures

Ta kategoria dotyczy błędów związanych z szyfrowaniem, haszowaniem, generowaniem losowości i ochroną danych w tranzycie oraz w spoczynku. Problem polega nie tylko na braku szyfrowania, ale też na użyciu słabych algorytmów, przewidywalnych wartości losowych czy niebezpiecznych implementacji kryptograficznych.

Cryptographic Failures (32 CWEs)

- **CWE-327 (Broken or Risky Cryptographic Algorithm)** — Używany jest przestarzały lub słaby algorytm kryptograficzny, który można łatwo złamać.
- **CWE-330 (Insufficiently Random Values)** — Generowane wartości losowe są przewidywalne, co osłabia bezpieczeństwo (np. tokenów).
- **CWE-331 (Insufficient Entropy)** — System nie ma wystarczającej losowości przy generowaniu danych kryptograficznych.
- **CWE-338 (Weak PRNG)** — Używany generator pseudolosowy nie jest kryptograficznie bezpieczny.
- **CWE-347 (Improper Verification of Cryptographic Signature)** — Podpisy cyfrowe nie są poprawnie weryfikowane, co pozwala na ich podrobienie.

Cryptographic Failures

- Klasyfikacja i oznaczanie danych
Najpierw trzeba ustalić, jakie dane aplikacja przetwarza, przechowuje i przesyła, a potem przypisać im poziomy wrażliwości. To pozwala dobrać odpowiednie zabezpieczenia do danych regulowanych prawnie, biznesowo krytycznych albo szczególnie poufnych.
- Zaufane implementacje kryptografii
Należy korzystać ze sprawdzonych bibliotek i gotowych implementacji algorytmów kryptograficznych, zamiast pisać własne mechanizmy. W kryptografii własne rozwiązania bardzo często prowadzą do błędów, które wyglądają poprawnie, ale są łamliwe.
- Minimalizacja ilości przechowywanych danych
Dane wrażliwe nie powinny być trzymane dłużej niż to konieczne; najlepiej usuwać je jak najszybciej albo zastępować tokenizacją lub skróceniem. Im mniej danych jest fizycznie zapisanych, tym mniej można stracić przy incydencie.
- Szyfrowanie danych w spoczynku
Wszystkie dane poufne powinny być szyfrowane na dysku lub w bazie danych. To ogranicza skutki kradzieży nośnika, kopii zapasowej albo nieautoryzowanego dostępu do magazynu danych.
- Dobieranie zabezpieczeń do klasy danych
Im wyższa klasyfikacja danych, tym mocniejsze powinny być stosowane kontrole bezpieczeństwa. To oznacza np. silniejsze uwierzytelnianie, ostrzejsze ograniczenia dostępu i bardziej rygorystyczne logowanie działań.

Injection

To podatność, w której nieufne dane użytkownika trafiają do interpretera, a ten wykonuje je jak polecenia albo fragmenty zapytania. Skutek bywa bardzo silny: od SQL Injection i command injection po XSS, LDAP injection czy code injection, czyli przejęcie logiki wykonywanej przez silnik pośredni.

Injection (37 CWEs)

- **CWE-20 (Improper Input Validation)** — Dane wejściowe nie są odpowiednio sprawdzane, co umożliwia ich nadużycie.
- **CWE-74 (Injection)** — Aplikacja interpretuje dane wejściowe jako polecenia lub kod.
- **CWE-77 (Command Injection)** — Dane użytkownika trafiają do poleceń systemowych i są wykonywane przez system operacyjny.
- **CWE-78 (OS Command Injection)** — Szczególny przypadek command injection umożliwiający wykonanie komend systemowych.
- **CWE-79 (Cross-Site Scripting, XSS)** — Aplikacja wstawia niesprawdzone dane do strony, co pozwala uruchomić złośliwy JavaScript.
- **CWE-89 (SQL Injection)** — Dane wejściowe wpływają na zapytania SQL, umożliwiając manipulację bazą danych.

Injection

- Używanie bezpiecznych API i ORM
Najlepszą metodą ochrony przed wstrzyknięciami jest całkowite oddzielenie danych od poleceń poprzez bezpieczne API lub ORM. Takie podejście umożliwia parametryzację zapytań i eliminuje konieczność ręcznego składania poleceń, minimalizując ryzyko SQL injection.
- Pozytywna walidacja po stronie serwera
Warto weryfikować dane wejściowe według zestawu dozwolonych wzorców, akceptując tylko przewidywane znaki i formaty. Choć nie daje to pełnej ochrony, ogranicza wprowadzanie niebezpiecznych danych i wspiera dalsze mechanizmy bezpieczeństwa.
- Unikanie dynamicznego składania zapytań w procedurach
Nawet procedury składowane mogą być podatne, jeśli używają dynamicznego SQL (np. EXECUTE IMMEDIATE, exec()) i łączą dane z kodem. Należy unikać takiego podejścia, bo ponownie miesza ono dane z logiką zapytania.

Insecure Design

To błędy na poziomie projektu, czyli „brak lub nieskuteczny projekt kontroli bezpieczeństwa”. Różnica względem błędów implementacji jest taka, że tutaj sama architektura albo model działania nie przewiduje odpowiedniej ochrony przed klasą ataku, więc nawet poprawne kodowanie nie usuwa źródła problemu.

Insecure Design (33 CWEs)

- **CWE-269 (Improper Privilege Management)** — System nie zarządza poprawnie poziomami uprawnień użytkowników.
- **CWE-434 (Unrestricted File Upload)** — Aplikacja pozwala przesyłać niebezpieczne pliki bez kontroli.
- **CWE-501 (Trust Boundary Violation)** — Dane przekraczają granice zaufania bez odpowiedniej weryfikacji.
- **CWE-602 (Client-Side Enforcement)** — Krytyczne sprawdzenia bezpieczeństwa wykonywane są tylko po stronie klienta.
- **CWE-841 (Improper Workflow Enforcement)** — Aplikacja nie wymusza poprawnej kolejności działań użytkownika.
- **CWE-1021 (UI Layer Restriction Failure)** — Interfejs użytkownika nie ogranicza poprawnie dostępu do funkcji lub danych.

Insecure Design

- Testy jednostkowe i integracyjne odporności na zagrożenia
Należy pisać testy, które sprawdzają odporność krytycznych przepływów na zidentyfikowane zagrożenia. Tworzenie scenariuszy użycia i nadużycia dla każdej warstwy pozwala wykryć luki zanim trafią do produkcji.
- Biblioteka bezpiecznych wzorców projektowych i komponentów
Warto utrzymywać zestaw sprawdzonych, „paved-road” komponentów i wzorców projektowych, które upraszczają implementację bezpiecznych funkcji. To skraca czas wytwarzania kodu i zmniejsza ryzyko powielania błędów w wielu miejscach aplikacji.
- Modelowanie zagrożeń dla krytycznych części aplikacji
Dla kluczowych obszarów, takich jak uwierzytelnianie, kontrola dostępu, logika biznesowa czy kluczowe przepływy danych, należy przeprowadzać modelowanie zagrożeń. To pozwala zidentyfikować potencjalne ataki i wprowadzić odpowiednie środki zaradcze już na etapie projektowania.
- Sprawdzanie poprawności na każdym poziomie aplikacji
Każda warstwa, od frontendu po backend, powinna zawierać mechanizmy walidacji i plausibility checks. To zmniejsza ryzyko, że złośliwe dane lub niepoprawne przepływy przejdą przez system niezauważone.

Authentication Failures

Ta kategoria dotyczy sytuacji, gdy system błędnie uznaje niewłaściwego użytkownika za prawidłowego albo pozwala zbyt łatwo obejść uwierzytelnianie. Zagrożenie obejmuje m.in. credential stuffing, brute force, słabe hasła, słabe odzyskiwanie konta, brak skutecznego MFA oraz problemy z sesjami i tokenami.

Authentication Failures (36 CWEs)

- **CWE-307 (Improper Restriction of Excessive Attempts)** — Brak ograniczeń liczby prób logowania umożliwia brute force.
- **CWE-384 (Session Fixation)** — Atakujący może narzucić identyfikator sesji ofierze.
- **CWE-521 (Weak Password Requirements)** — System dopuszcza słabe hasła.
- **CWE-613 (Insufficient Session Expiration)** — Sesje użytkowników wygasają zbyt późno lub wcale.
- **CWE-640 (Weak Password Recovery)** — Proces odzyskiwania hasła jest łatwy do przejęcia.
- **CWE-798 (Hard-coded Credentials)** — Dane logowania są zapisane na stałe w kodzie.

Authentication Failures

- Wymuszanie uwierzytelniania wieloskładnikowego (MFA)
Tam, gdzie to możliwe, należy stosować MFA, aby chronić konta przed atakami typu credential stuffing, brute force i ponownym użyciem skradzionych danych uwierzytelniających. Nawet jeśli hasło zostanie przechwycone, atakujący nie uzyska dostępu bez drugiego czynnika.
- Zachęcanie do użycia menedżerów haseł
Użytkownicy powinni być wspierani w stosowaniu menedżerów haseł, co ułatwia tworzenie silnych, unikalnych haseł i zmniejsza ryzyko powtarzania tych samych haseł w wielu usługach.
- Brak domyślnych poświadczeń
Systemy nie powinny być dostarczane z predefiniowanymi hasłami, zwłaszcza dla kont administracyjnych. Domyślne dane uwierzytelniające są łatwym celem atakujących i muszą być zmienione przed pierwszym użyciem.
- Sprawdzanie słabych haseł
Nowe i zmieniane hasła powinny być weryfikowane przeciwko listom najczęściej używanych lub najslabszych haseł, np. top 10,000. To ogranicza ryzyko, że użytkownik wybierze łatwe do odgadnięcia hasło.

Software or Data Integrity Failures

Chodzi o brak pewności, że kod albo dane pochodzą z zaufanego źródła i nie zostały podmienione. Problem pojawia się przy zależnościach z niepewnych repozytoriów, niezweryfikowanych aktualizacjach, niekontrolowanych pipeline'ach CI/CD oraz przy deserializacji danych, które atakujący może zmienić.

Software or Data Integrity Failures (14 CWEs)

- **CWE-345 (Insufficient Verification of Data Authenticity)** — System nie sprawdza, czy dane są autentyczne i niezmienione.
- **CWE-494 (Download Without Integrity Check)** — Kod lub pliki są pobierane bez sprawdzania ich integralności.
- **CWE-502 (Deserialization of Untrusted Data)** — Aplikacja przetwarza niezweryfikowane dane jako obiekty, co może prowadzić do wykonania kodu.
- **CWE-829 (Untrusted Control Sphere Inclusion)** — Aplikacja łąduje kod lub funkcje z niezaufanego źródła.
- **CWE-830 (Untrusted Web Source Inclusion)** — Zewnętrzne zasoby webowe są dołączane bez weryfikacji.

Software or Data Integrity Failures

- Weryfikacja źródła i integralności danych za pomocą podpisów cyfrowych
Dane lub oprogramowanie powinny być podpisane cyfrowo, aby upewnić się, że pochodzą z oczekiwanego źródła i nie zostały zmodyfikowane. Takie podejście chroni przed wprowadzeniem złośliwego kodu lub manipulacją podczas transmisji lub dystrybucji.
- Korzystanie tylko z zaufanych repozytoriów bibliotek i zależności
Wszystkie pakiety, np. z npm czy Maven, powinny pochodzić z wiarygodnych źródeł. Dla środowisk wysokiego ryzyka warto rozważyć wewnętrzne repozytorium „known-good”, w którym pakiety są zweryfikowane przed użyciem.
- Proces przeglądu kodu i konfiguracji
Każda zmiana w kodzie lub konfiguracji powinna przechodzić przez formalny proces przeglądu. Dzięki temu zmniejsza się ryzyko przypadkowego lub celowego wprowadzenia złośliwych elementów do pipeline'u.

Security Logging and Alerting Failures

To brak albo słabość logowania, monitoringu i alertowania, przez co ataki są późno wykrywane albo w ogóle nie są zauważane. Zagrożenie polega na tym, że zdarzenia bezpieczeństwa nie trafiają do logów, logi są niepełne, można je modyfikować, albo nikt ich aktywnie nie obserwuje.

Security Logging and Alerting Failures (5 CWEs)

- **CWE-117 (Improper Output Neutralization for Logs)** — Atakujący może manipulować logami, wstrzykując do nich dane.
- **CWE-223 (Omission of Security-relevant Information)** — Brak logów/informacji potrzebnych do identyfikacji ataku lub oceny bezpieczeństwa działania.
- **CWE-221 (Information Loss or Omission)** - Brak lub błędne logowanie danych bezpieczeństwa, co utrudnia decyzje i analizę.
- **CWE-532 (Insertion of Sensitive Information into Log File)** — W logach zapisywane są wrażliwe dane.
- **CWE-778 (Insufficient Logging)** — Logowanie jest niewystarczające do wykrycia incydentów.

Security Logging and Alerting Failures

- Logowanie zdarzeń bezpieczeństwa z kontekstem użytkownika
Wszystkie błędy logowania, kontroli dostępu i walidacji po stronie serwera powinny być logowane z wystarczającym kontekstem użytkownika. Logi muszą być przechowywane na tyle długo, aby możliwa była analiza kryminalistyczna po opóźnieniu zdarzeń.
- Alertowanie przy podejrzanych zachowaniach
System powinien generować alerty, jeśli aplikacja lub użytkownicy zachowują się nietypowo. Programiści powinni znać wytyczne dotyczące takich alertów, a jeśli to możliwe, warto korzystać z gotowych systemów do monitorowania i powiadamiania.
- Skuteczne przypadki użycia monitorowania i alertowania
DevSecOps i zespoły bezpieczeństwa powinny opracować playbooki i scenariusze, które pozwalają wykrywać i reagować na podejrzane działania szybko i skutecznie.
- Wykorzystanie „honeytokens”
Wstawienie pułapek dla atakujących, np. w bazie danych czy fałszywych kont użytkowników, generuje logi przy próbie ich użycia. Ponieważ prawdziwi użytkownicy ich nie używają, alerty praktycznie nie mają fałszywych pozytywów.

Mishandling of Exceptional Conditions

Ta kategoria dotyczy nieprawidłowego obchodzenia się z wyjątkami, błędami i nietypowymi stanami programu. Ryzyko polega na tym, że aplikacja nie wykrywa sytuacji awaryjnej, nie reaguje na nią właściwie albo wpada w stan nieprzewidywalny, co może prowadzić do crashy, wycieków informacji lub obejścia logiki bezpieczeństwa.

Mishandling of Exceptional Conditions (24 CWEs)

- **CWE-209 (Error Message Information Leak)** — Komunikaty błędów ujawniają zbyt dużo informacji o systemie.
- **CWE-391 (Unchecked Error Condition)** — Błędy nie są sprawdzane ani obsługiwane.
- **CWE-476 (NULL Pointer Dereference)** — Program odwołuje się do pustego wskaźnika, co może prowadzić do crasha.
- **CWE-636 (Not Failing Securely)** — System w razie błędu przechodzi w stan niebezpieczny zamiast bezpiecznego.
- **CWE-703 (Improper Handling of Exceptional Conditions)** — Aplikacja nie radzi sobie poprawnie z wyjątkami.
- **CWE-755 (Improper Handling of Exceptional Conditions)** — Ogólna kategoria błędów w obsłudze sytuacji wyjątkowych.

Mishandling of Exceptional Conditions

- Planowanie obsługi wyjątków i przewidywanie najgorszego
Aplikacja powinna przewidywać potencjalne błędy i planować ich obsługę z wyprzedzeniem. Dzięki temu system nie zostaje zaskoczony nieprzewidzianymi sytuacjami, co zmniejsza ryzyko awarii lub podatności.
- Globalny handler wyjątków
Warto mieć centralny mechanizm obsługi wyjątków, który przechwyci wszelkie nieprzewidziane błędy. Zapewnia to, że żaden błąd nie pozostanie niezarejestrowany i że użytkownik otrzyma spójny komunikat o problemie.
- Monitoring i obserwowalność powtarzających się błędów
Narzędzia monitorujące powinny wykrywać powtarzające się błędy lub wzorce wskazujące na atak, np. boty wykorzystujące słabe punkty w obsłudze wyjątków. System może wtedy uruchomić odpowiedź obronną lub blokadę ataku.
- Ograniczenia i limitowanie zasobów
Stosowanie rate limiting, limitów zasobów, throttlingu i innych ograniczeń zmniejsza prawdopodobieństwo wystąpienia błędów z powodu nadmiernego obciążenia. To chroni przed DoS, atakami brute-force oraz nadmiernymi kosztami w chmurze.